LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 6
Student Name: Gorapalli Sravya
Roll No.: A24126552257
Date: 26-08-2026

---

1. TITLE

Develop a Mini Student Information Application Using Express and Node.js

2. AIM

To develop a mini web application using Node.js and Express.js that accepts student information through an HTML form and displays the submitted details.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the basics of Node.js and Express.js.
- To create a simple Express server.
- To implement GET and POST routes.
- To accept form data from a web page.
- To process the submitted data using Node.js and display it as a response.

4. THEORY

Node.js is a JavaScript runtime environment that allows JavaScript code to run outside the browser, commonly used to build server-side applications.

Express.js is a lightweight web framework for Node.js that simplifies the creation of routes, middleware and web servers. In this application, Express is used to handle client requests: a GET route displays the student form, and a POST route receives the submitted data.

The middleware express.urlencoded() is used to parse form data so it can be accessed through req.body. The basic flow of the application is: Browser -> Express Server -> Node.js Processing -> Response -> Browser.

5. ALGORITHM / PROCEDURE

1. Create a new Node.js project folder.
2. Initialize the project using npm init.
3. Install Express using npm install express.
4. Import the Express module and create an Express application.

5. Configure middleware (express.urlencoded) to read form data.
6. Create a GET route to display the student registration form.
7. Create a POST route to receive the submitted student details.
8. Extract the name, roll number and marks from the request body.
9. Generate and send a response containing the submitted student details.
10. Start the Express server on port 3000.
11. Open the application in a web browser, fill the form and submit it.
12. Verify the displayed output against the expected output.

6. PROGRAM

server.js

```
const express = require("express");
const app = express();
const PORT = 3000;

// Middleware to read form data
app.use(express.urlencoded({ extended: true }));

// Display student form
app.get("/", (req, res) => {
  res.send(`
    <h1>Student Information System</h1>
    <form action="/student" method="POST">
      <label>Name:</label>
      <input type="text" name="name" required>
      <br><br>
      <label>Roll No:</label>
      <input type="text" name="rollno" required>
      <br><br>
      <label>Marks:</label>
      <input type="number" name="marks" required>
      <br><br>
      <button type="submit">Submit</button>
    </form>
  `);
});

// Process submitted student details
app.post("/student", (req, res) => {
  const name = req.body.name;
  const rollno = req.body.rollno;
  const marks = req.body.marks;

  res.send(`
    <h1>Student Details</h1>
    <p><strong>Name:</strong> ${name}</p>
    <p><strong>Roll No:</strong> ${rollno}</p>
    <p><strong>Marks:</strong> ${marks}</p>
    <br>
    <a href="/">Go Back</a>
  `);
});

// Start the server
```

```
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

7. CODE EXPLANATION

Tag / Code	Explanation
require("express")	Imports the Express module
express()	Creates an Express application
express.urlencoded()	Allows the server to read submitted form data
app.get("/")	Handles GET requests and displays the student form
app.post("/student")	Handles the submitted student information
req.body	Contains the submitted form data
res.send()	Sends a response back to the browser
app.listen()	Starts the server on the specified port

8. TEST CASES AND EXECUTION DATA

The server was started and the form was submitted with several sets of test data; the results were recorded below.

Test Case	Description	Expected Result	Actual Result	Status
TC-01	Submit form: Rahul, 101, 85	Student details displayed correctly	Student details displayed correctly	Pass
TC-02	Submit form: Priya, 102, 92	Student details displayed correctly	Student details displayed correctly	Pass
TC-03	Submit form: Arjun, 103, 67	Student details displayed correctly	Student details displayed correctly	Pass
TC-04	Submit form: Kiran, 104, 0	Student details displayed correctly	Student details displayed correctly	Pass
TC-05	Submit form with Name left empty	Browser should require the Name field	Browser required the Name field	Pass

9. OUTPUT

Output: Server Execution and Form Submission

[Insert screenshots of the terminal showing the server running, the student form, and the submitted student details page.]

10. RESULT

Thus, a mini Student Information Application was successfully developed using Node.js and Express.js. GET and POST requests were implemented, student form data was processed successfully, and the application was verified using multiple test cases.